

```
package adt;

/*@Author:Ganesha*/

import java.util.Iterator;
import java.util.function.Predicate;

public class MyArrayList<T> implements Iterable<T> {

    private T[] data;
    private int size;

    @SuppressWarnings("unchecked")
    public MyArrayList() {
        data = (T[]) new Object[10]; // Default initial capacity
        size = 0;
    }

    public void add(T element) {
        if (size == data.length) {
            resize();
        }
        data[size++] = element;
    }

    public T get(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException();
        }
        return data[index];
    }

    public T remove(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("Index out of bounds: " + index);
        }
        T removedElement = data[index];
        // Shift elements to the left
        for (int i = index; i < size - 1; i++) {
            data[i] = data[i + 1];
        }
    }
}
```

x) ;

```
        data[--size] = null; // Nullify the last element
        return removedElement; // Return the removed element
    }

    public int size() {
        return size;
    }

    public void clear() {
        size = 0;
    }

    private void resize() {
        @SuppressWarnings("unchecked")
        T[] newData = (T[]) new Object[data.length * 2];
        System.arraycopy(data, 0, newData, 0, data.length);
        data = newData;
    }

    // Implement Iterable
    @Override
    public Iterator<T> iterator() {
        return new Iterator<T>() {
            private int currentIndex = 0;

            @Override
            public boolean hasNext() {
                return currentIndex < size;
            }

            @Override
            public T next() {
                return data[currentIndex++];
            }
        };
    }

    // Add multiple items from another MyArrayList
    public void addAll(MyArrayList<T> other) {
        for (int i = 0; i < other.size(); i++) {
            this.add(other.get(i));
        }
    }
}
```



```
    }  
}  
  
// Remove items matching a condition  
public boolean removeIf(Predicate<T> condition) {  
    boolean removed = false;  
  
    for (int i = 0; i < size; i++) {  
        if (condition.test(data[i])) {  
            for (int j = i; j < size - 1; j++) {  
                data[j] = data[j + 1];  
            }  
            data[--size] = null;  
            i--;  
            removed = true;  
        }  
    }  
    return removed;  
}  
}
```

